CUDA & Performance Engineering

Some slides adapted from CSE 554 by Baris Kasikci

Administrativa

- HW9 Test Case due tonight
- Survey open
- Quiz Wednesday
- Rest of quarter plan

Agenda

- Performance Engineering, CPU example
- CUDA
- Writing inefficient CUDA code
- Writing efficient CUDA code with profiling
- Takeaways

A quote

“There is no doubt that the grail of efficiency leads to abuse. Programmers waste enormous amounts of time thinking about, or worrying about, the speed of noncritical parts of their programs, and these attempts at efficiency actually have a strong negative impact when debugging and maintenance are considered. We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil.

Yet we should not pass up our opportunities in that critical 3%. A good programmer will not be lulled into complacency by such reasoning, he will be wise to look carefully at the critical code; but only after that code has been identified. It is often a mistake to make a priori judgments about what parts of a program are really critical, since the universal experience of programmers who have been using measurement tools has been that their intuitive guesses fail.”

— Donald Knuth, 1974

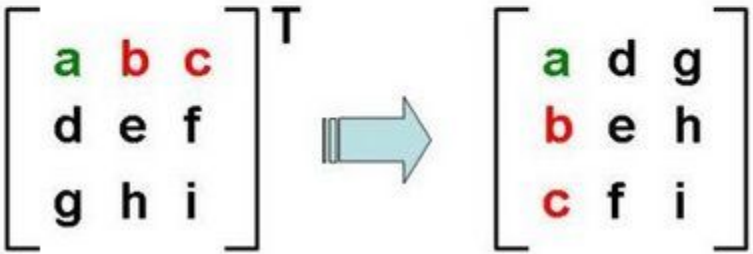
Performance Engineering

Building software that *will* need to run fast

Key pieces to making this work:

- Architectural knowledge
- Performance modeling
- Profiling bottlenecks
- Performance monitoring

Example: CPU Math Libraries



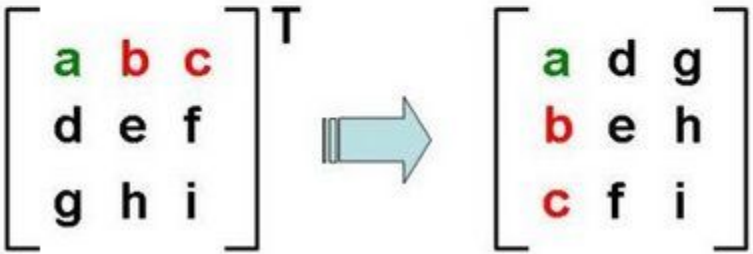
Goal: transpose a 4x4 matrix, quickly.

Code:

```
for (row in 0..4):
    for (column in 0..4):
        output[column][row] = input[row][column]
```

Works great!

Example: CPU Math Libraries



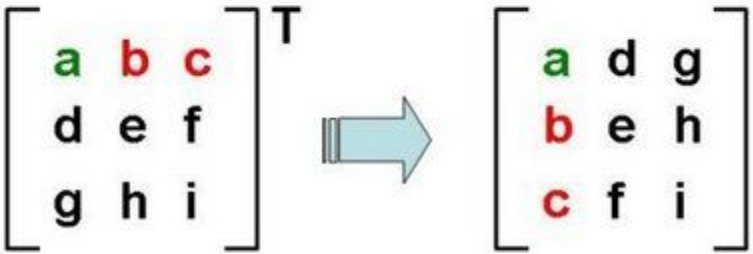
Goal: transpose a 5x5 matrix, quickly.

Code:

```
for (row in 0..5):
    for (column in 0..5):
        output[column][row] = input[row][column]
```

Runs slowly…
We'd expect (25/16) = 1.6x slowdown, but we see 4x slowdown!
What happened?

Example: CPU Math Libraries

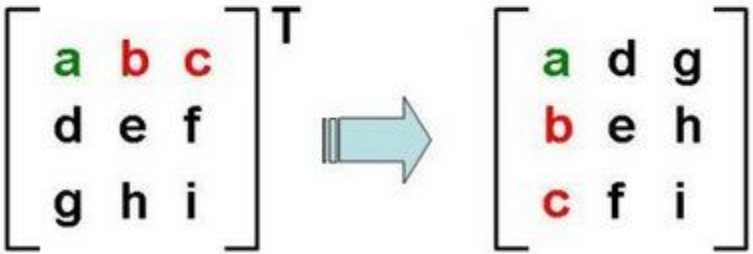


Bring out the profiler…

```
transpose_4x4 spends 10 cycles reading from memory

transpose_5x5 spends 30 cycles reading from memory
```

Example: CPU Math Libraries



Bring out the assembly…

```
transpose_4x4:
    load input[0][0], %rax
    load input[0][1], %rbx
    …
    store %rax, output[0][0]
    store %rbx, output[1][0]
    …
```

What happens when we transpose bigger matrices?
We run out of registers!

Example: CPU Math Libraries

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}^T \Rightarrow \begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix}$$

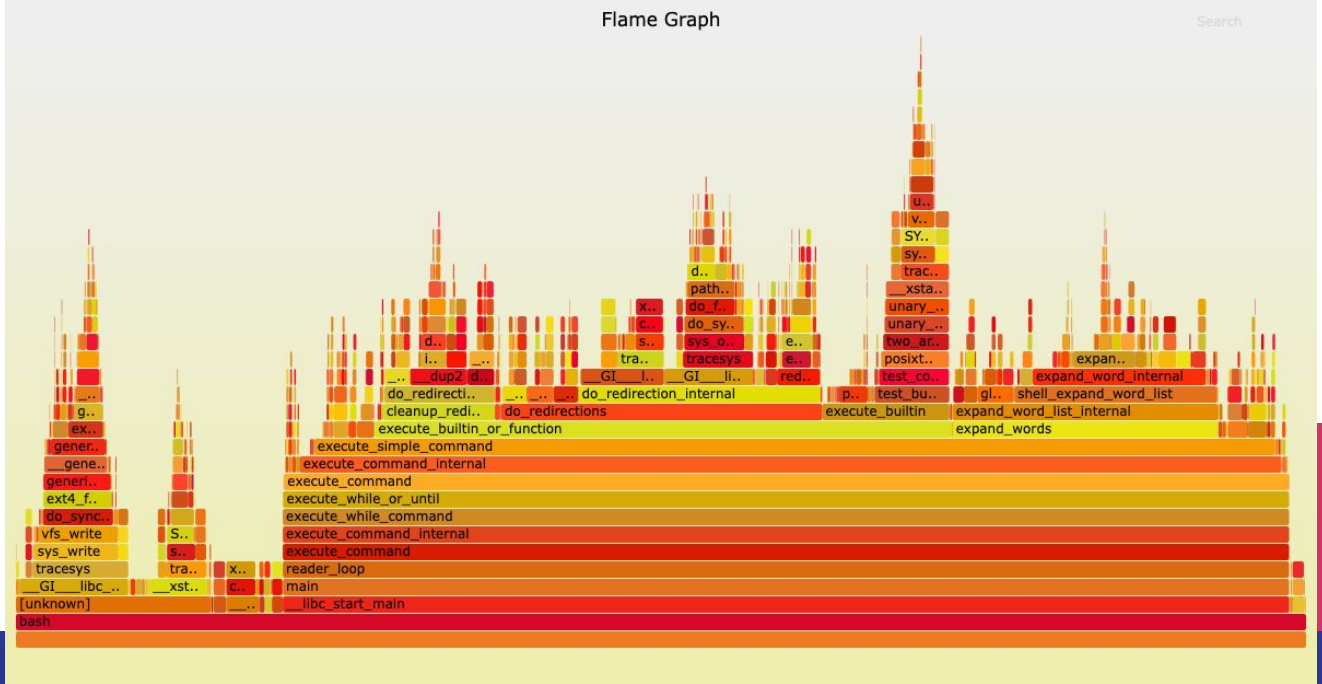
Bring out the assembly…

```
transpose_5x5:
    load some inputs
    store some outputs
    load rest of inputs
    store rest of outputs
```

4 round trips to memory instead of 2

CPU Profiling in Practice

Profilers help you find which functions take a long time to run


Flame Graph
Search
bash
[unknown]
__libc_start_main
main
reader_loop
execute_command
execute_command_internal
execute_while_command
execute_while_or_until
execute_command
execute_command_internal
execute_simple_command
execute_builtin_or_function
cleanup_redi..
do_redirecti..
do_redirections
do_redirection_internal
execute_builtin
expand_words
expand_word_list_internal
shell_expand_word_list
expand_word_internal
expan..
posixt..
test_co..
test_bu..
two_ar..
__xsta..
unary_..
unary_..
trac..
sy..
SY..
v..
u..
path..
do_f..
do_sy..
sys_o..
tracesys
__GI___li..
__GI___l..
__dup2
__GI___libc_..
__xst..
execute_command
execute_simple_command
execute_command_internal
execute_command
execute_while_or_until
execute_while_command
execute_command_internal
execute_command
reader_loop
main
__libc_start_main
ext4_f..
do_sync..
vfs_write
sys_write
tracesys
generi..
__gene..
gener..
ex..
g..
S..
s..
tra..
x..
c..
__..
d..
i..
_..
tra..
s..
c..
x..
d..
e..
e..
red..
p..
gl..
d..
path..
do_f..
do_sy..
sys_o..
tracesys
__GI___li..
__GI___l..
__dup2
d..
_..
_..
_

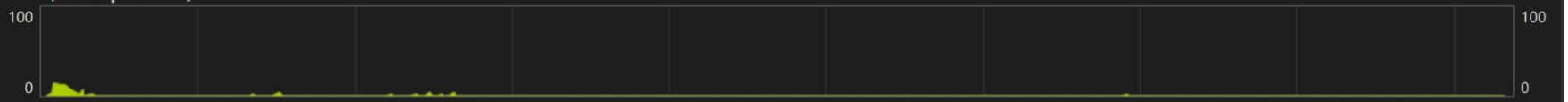
Report20220....diagsession* | Report2022041...4.diagsession

Output | Zoom In | Reset Zoom | Clear Selection

Diagnostics session: 1:57 minutes

12.5s | 25s | 37.5s | 50s | 1:02.5min | 1:15min | 1:27.5min | 1:40min | 1:52.5m

CPU (% of all processors)

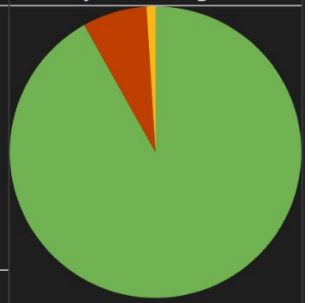


Open details... | Categories | Filter

Top Functions

Function Name	Total CPU [unit, %]	Self CPU [unit, %]
[External Call] ntdll.dll!0x00007ff85d342651	1570 (97.27%)	680 (42.13%)
[External Call] Microsoft.EntityFrameworkCore.Infrastructure.DatabaseFacade.EnsureCreated()	507 (31.41%)	371 (22.99%)
[External Call] Newtonsoft.Json.JsonSerializer.Deserialize<T>(Newtonsoft.Json.JsonReader)	111 (6.88%)	111 (6.88%)
[External Call] microsoft.extensions.hosting.abstractions.dll!0x00007ff854ed6573	737 (45.66%)	104 (6.44%)
[External Call] Microsoft.EntityFrameworkCore.DbContext.ctor(Microsoft.EntityFrameworkCore.DbContextOptions)	71 (4.40%)	71 (4.40%)

Top Five Categories



Kernel : 91.9% (1483)
Json : 7.1% (115)
Other : 1.0% (16)

Hot Path

Function Name	Total CPU [unit, %]	Self CPU [unit, %]
Scenarios (PID: 19176)	1614 (100.00%)	3 (0.19%)
[System Code] ntdll.dll!0x00007ff85d342651	1570 (97.27%)	680 (42.13%)
Scenarios.Program.Main(System.String[])	890 (55.14%)	0 (0.00%)

GPU programming is full of these pitfalls

- Running out of registers
- Memory bank conflicts
- Underutilized warps / divergence issues

GPUs

SIMD

- HW2 was meant to get you to program in this style

CUDA

The programming language for NVIDIA GPUs

Developed by NVIDIA, compiled by “nvcc” (nvidia c compiler)

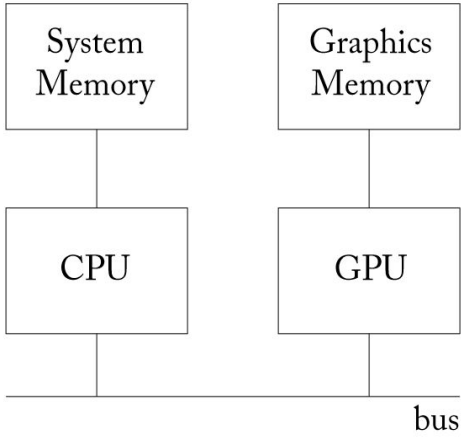
- nvcc is closed source, so is GPU ISA, so we don’t get to see ASM!

Built on top of C/C++

- Host (cpu) code is forwarded to gcc, launches “kernels”
- Device (gpu) code is compiled by nvcc

CUDA: Getting Full Control

- Memory allocation and deallocation
  - cudaMalloc -> device memory allocation
  - cudaMallocHost -> pinned host memory allocation
  - cudaFree -> free memory
- Memory movement and setting
  - cudaMemcpy -> synchronous copy
  - cudaMemcpyAsync -> asynchronous copy
  - cudaMemset -> synchronous set
  - cudaMemsetAsync -> asynchronous set



(a) System with discrete GPU

CUDA kernels

- Declaring a kernel
  - __global__ void kernel_name (args…)
- Declaring a device “helper” function
  - __device__ T helper_name (args …)
- Args are on the host
  - Pointers to device memory also reside in the host
- Inside a kernel, args (basic types, e.g. Int) can be used and device pointers can be dereferenced.

Launching a kernel

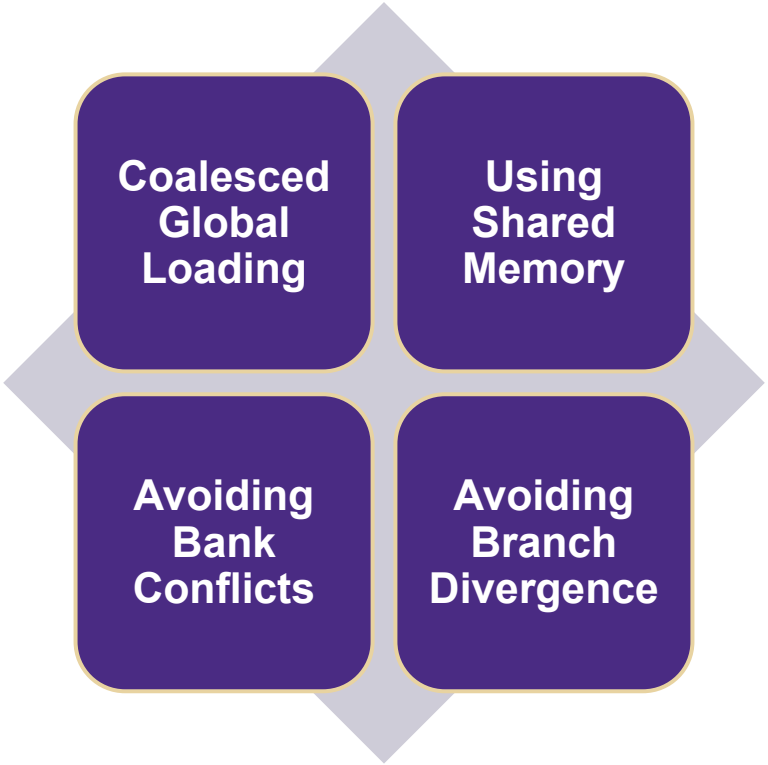
- Defining block shapes
  - dim3 block(x,y,z)
- Defining thread shapes (threads per block)
  - dim3 thread (x,y,z)
- Launching kernels
  - kernel_name<<<block, thread>>> (args);

Example: Vector Sum

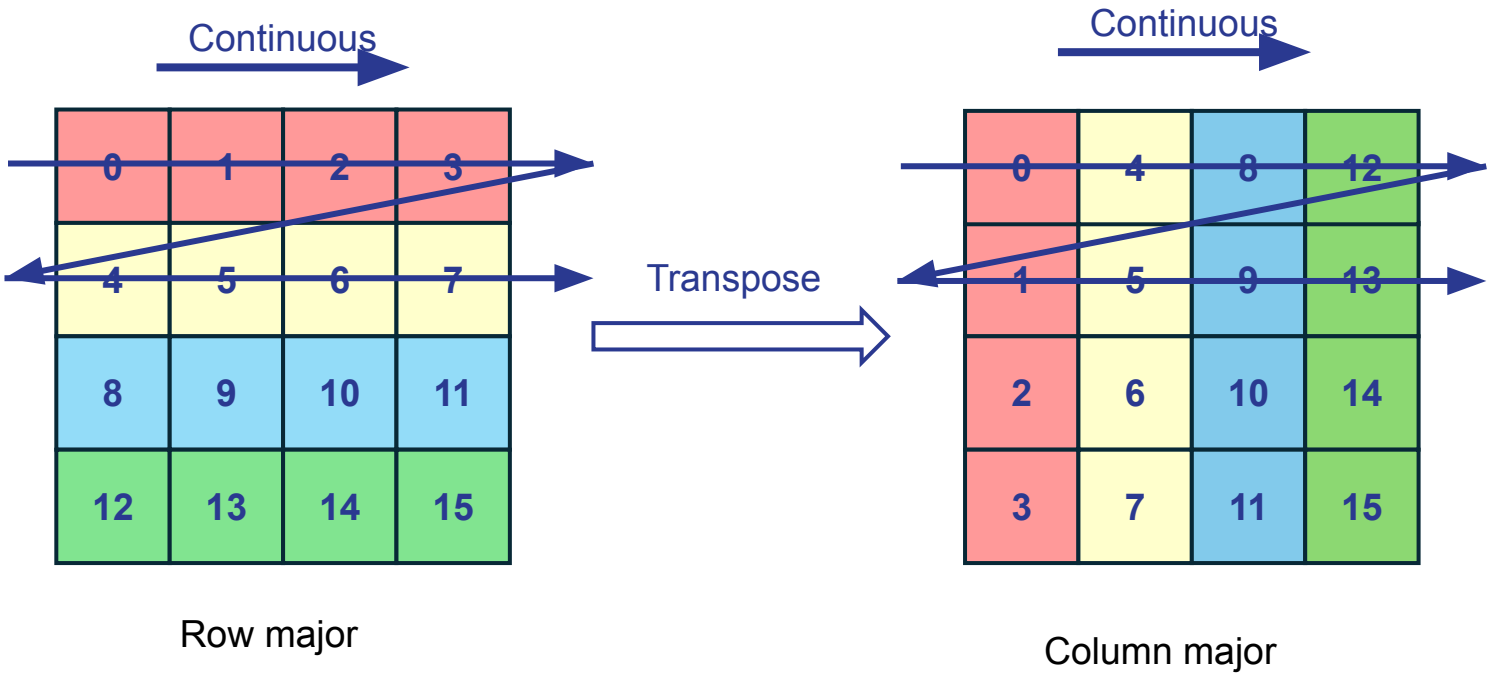
Notice:

- cuda malloc, memcpy
- d_ prefix for device pointers, h_ for host pointers
- rounding up block count

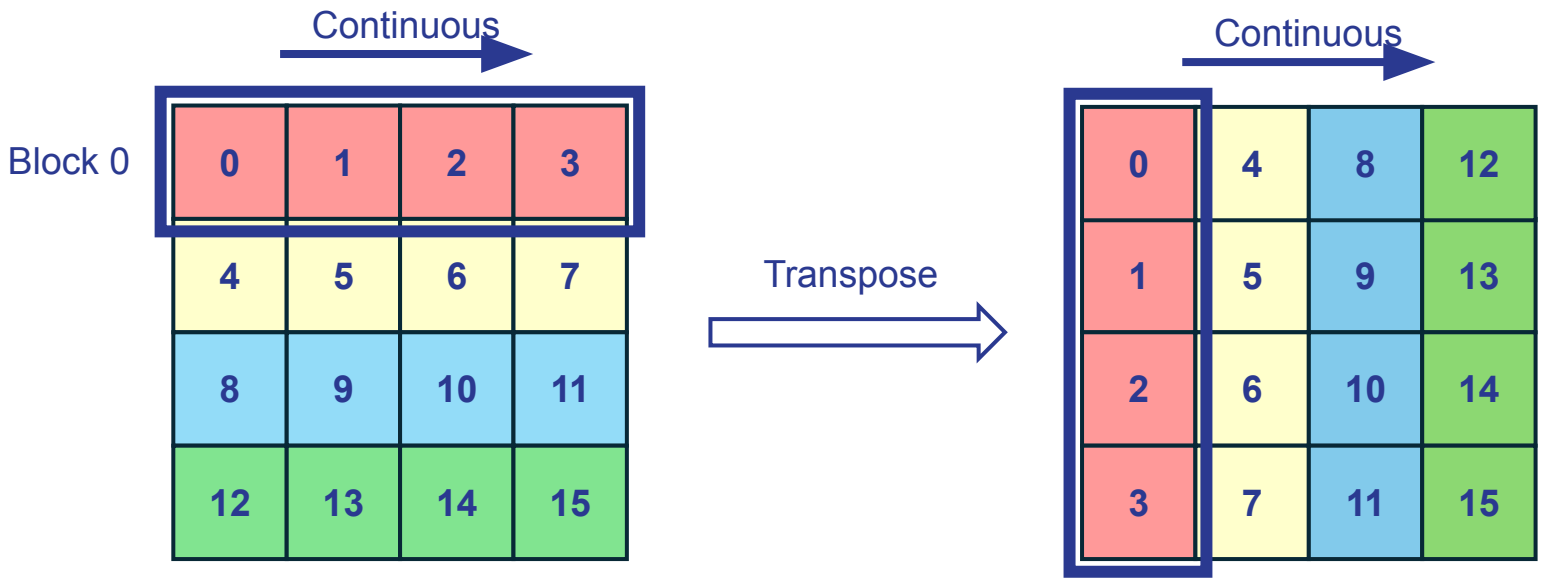
How to write fast kernels?



Matrix Transpose Kernel

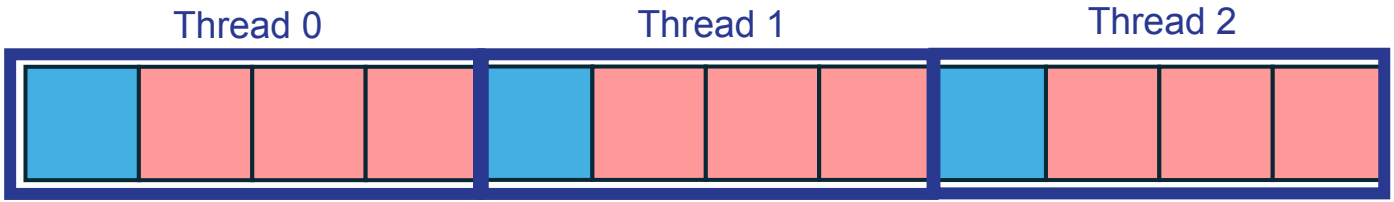


Transpose V1: Row-Wise Partitioning



https://developer.nvidia.com/blog/efficient-matrix-transpose-cuda-cc/

Transpose V1: ~3 ms



Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	1,146,880	Branch Efficiency [%]	100
Branch Instructions Ratio [%]	0.10	Avg. Divergent Branches	0

Uncoalesced Global Accesses
Est. Speedup: 86.35%

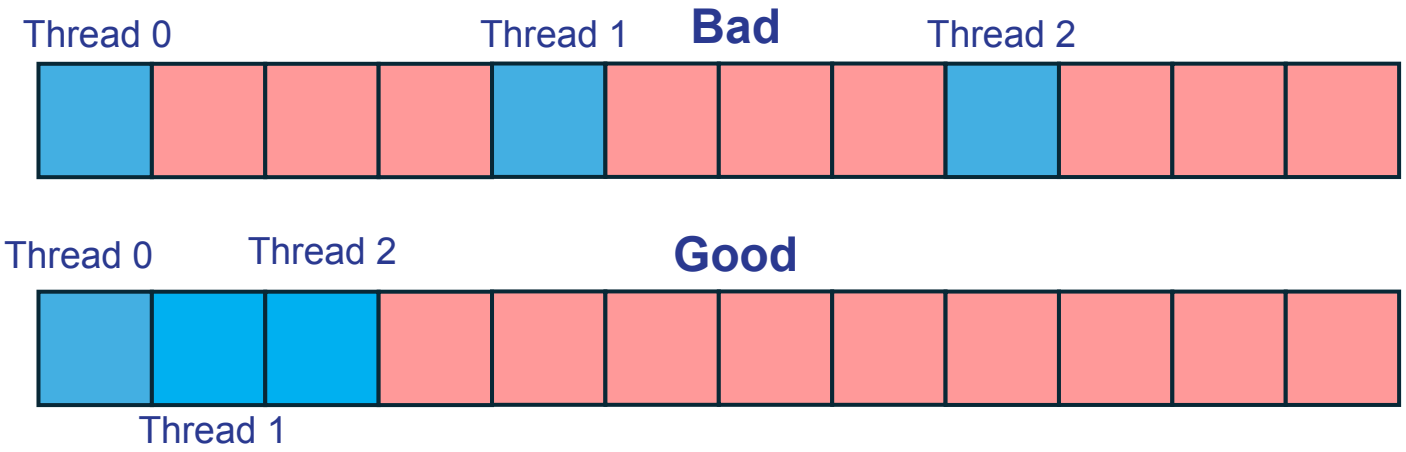
This kernel has uncoalesced global accesses resulting in a total of 117440512 excessive sectors (88% of the total 134217728 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
0x7f00fb99f990 in transpose	14,680,064	13
0x7f00fb99f970 in transpose	14,680,064	13
0x7f00fb99f960 in transpose	14,680,064	13
0x7f00fb99f920 in transpose	14,680,064	13
0x7f00fb99f910 in transpose	14,680,064	13

Transpose V2: Global memory coalescing

- Inside one warp, if the memory accesses are contiguous, the memory access is coalesced (batched)
- Data can be retrieved from global memory in one or a few transactions



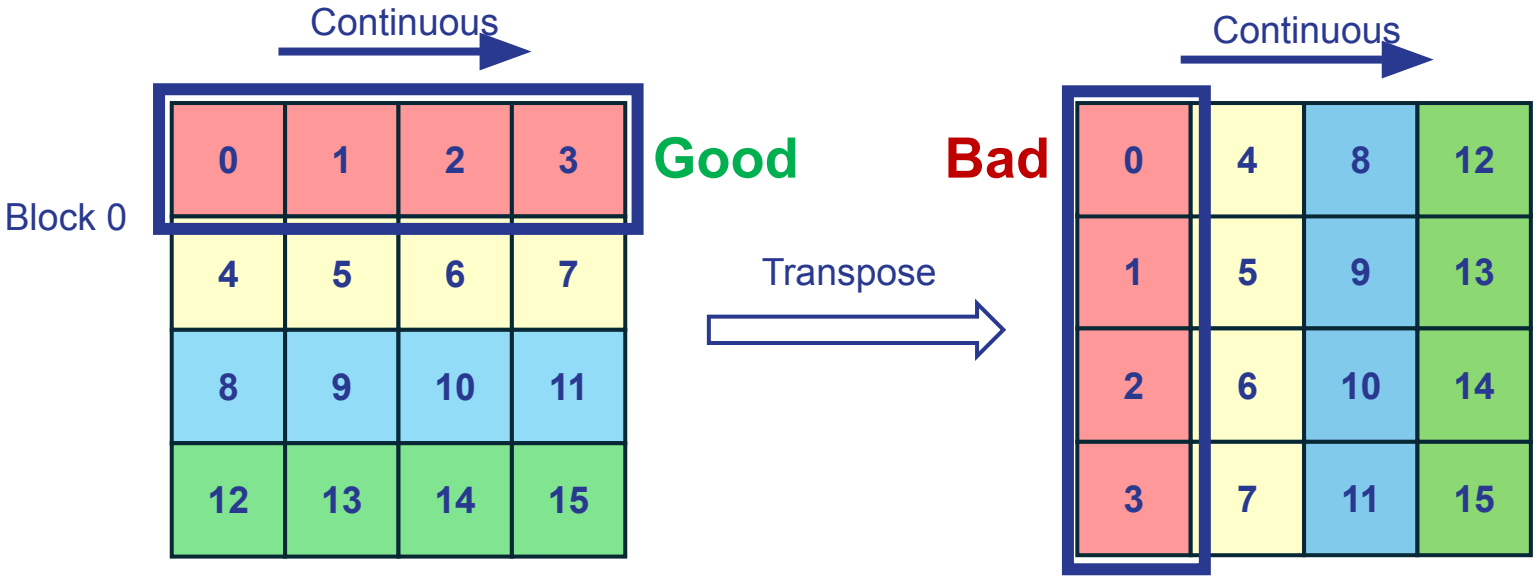
Transpose V2:1.40 ms: Uncoalesced Writes to the Output Matrix

Uncoalesced Global Accesses
Est. Speedup: 77.52%

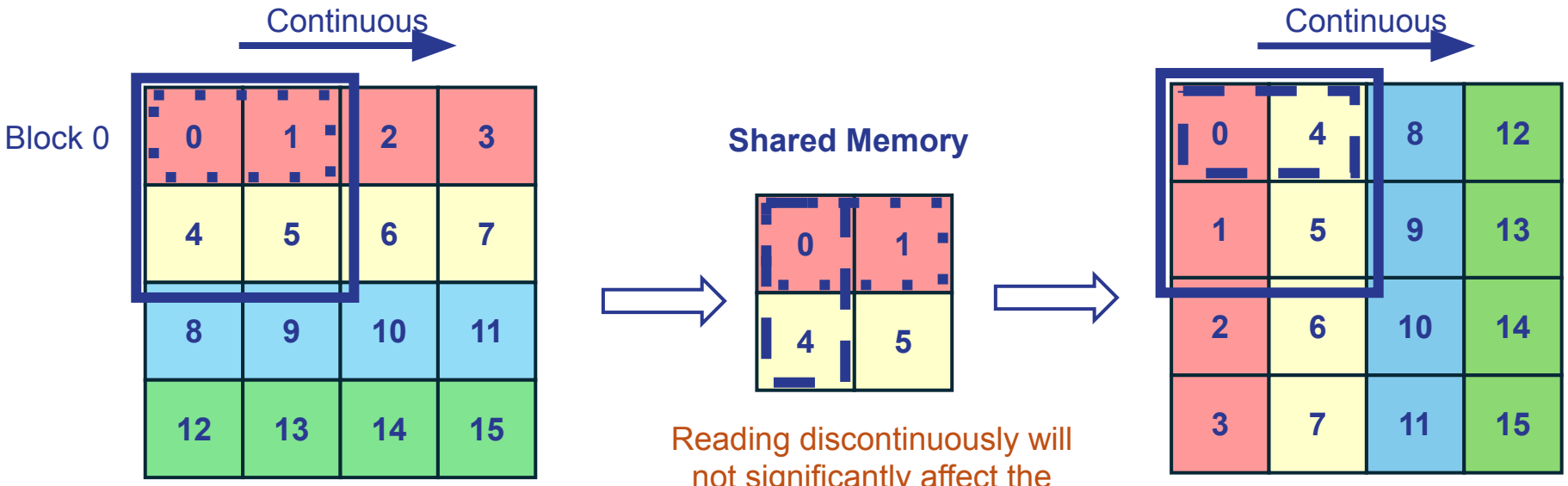
This kernel has uncoalesced global accesses resulting in a total of 58720256 excessive sectors (78% of the total 75497472 sectors). Check the L2 Theoretical Sectors Global Excessive table for the primary source locations. The CUDA Programming Guide has additional information on reducing uncoalesced device memory accesses.

L2 Theoretical Sectors Global Excessive

Location	Value	Value (%)
0x7f6fc799f720 in transpose	58,720,256	100
0x7f6fc799f6d0 in transpose	0	0



Transpose V3: Tilewise partitioning



Reading discontinuously will not significantly affect the shared memory performance (shared memory is fast)

Shared Memory Allocation

- Static allocation
  - Inside the kernel, call
    - `__shared__ float f_array[10];`
  - Easier to use
  - Fixed size, up to 48 KB
- Dynamic allocation
  - Inside the kernel call
    - `extern __shared__ int shared_mem[];`
      - No size specified yet; this will be done on the host
  - When launching the kernel, call
    - `my_kernel<<<grid, block, shared_mem_size_in_bytes>>>`
  - Above 48KB and up to 228 KB
    - `cudaFuncSetAttribute(my_kernel, cudaFuncAttributeMaxDynamicSharedMemorySize, desiredSizeInBytes);`

Transpose V3 312 us: Bank Conflict

Memory Workload Analysis Memory Chart

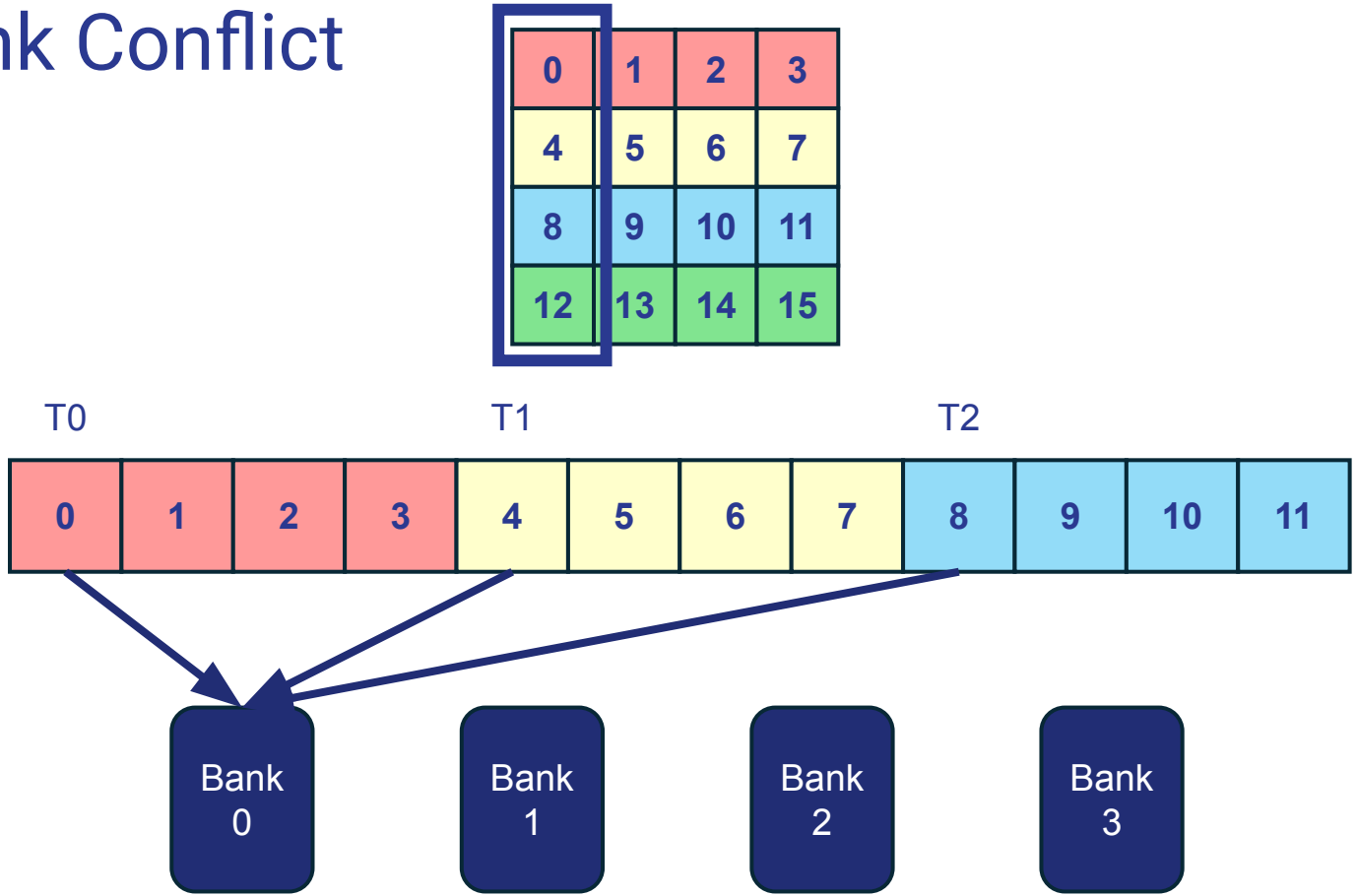
Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Tbyte/s]	2.13	Mem Busy [%]	48.15
L1/TEX Hit Rate [%]	0	Max Bandwidth [%]	63.53
L2 Hit Rate [%]	50.95	Mem Pipes Busy [%]	13.83
L2 Compression Input Sectors	8,389,199	L2 Compression Success Rate [%]	0
L2 Compression Ratio [%]	0	-	-

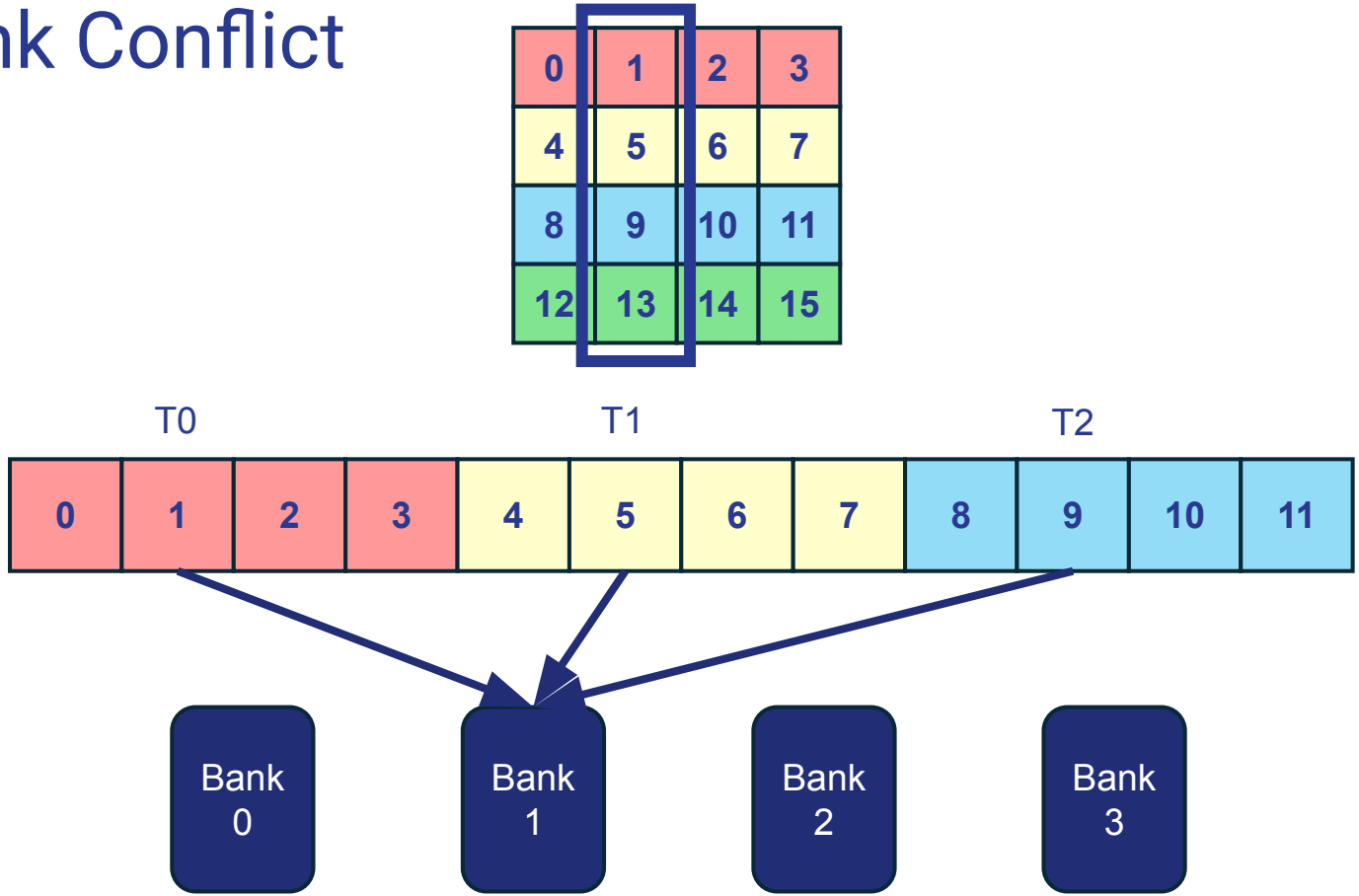
Low Compression Rate
Est. Speedup: 22.20%
Out of the 268454368.0 bytes sent to the L2 Compression unit only 0.00% were successfully compressed. To increase this success rate, consider marking only those memory regions as compressible that contain the most zero values and/or expose the most homogeneous values.

Shared Load Bank Conflicts
Est. Speedup: 43.79%
The memory access pattern for shared loads might not be optimal and causes on average a 33.0 - way bank conflict across all 524288 shared load requests.This results in 15197160 bank conflicts, which represent 87.87% of the overall 17294312 wavefronts for shared loads. Check the Source Counters section for uncoalesced shared loads.

Bank Conflict

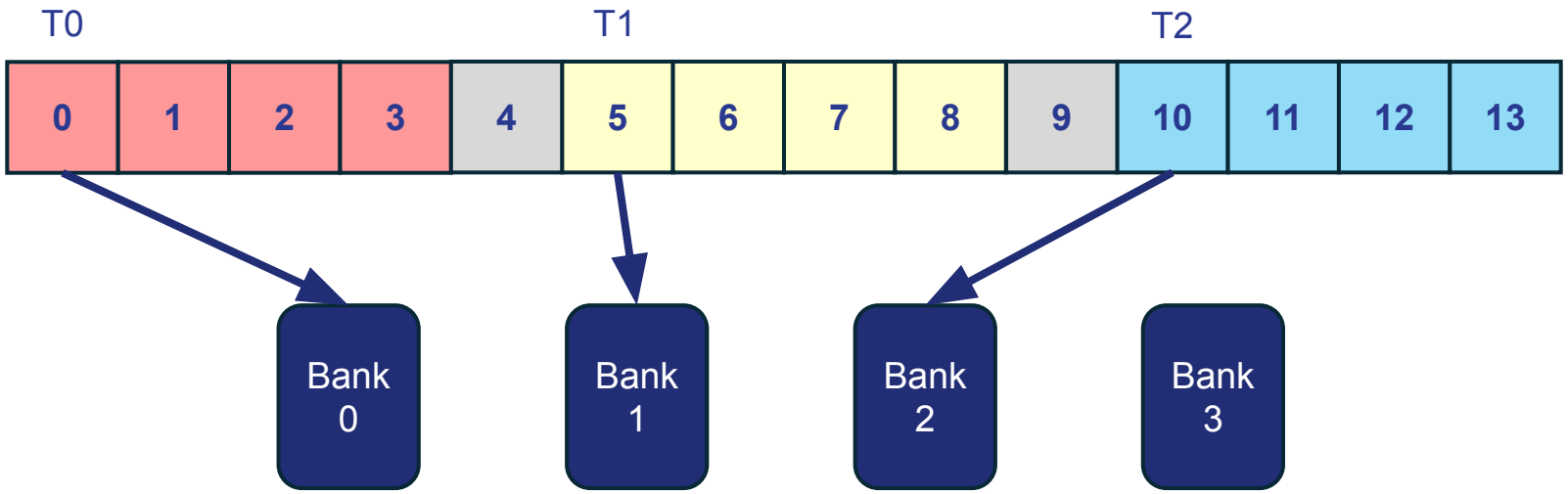


Bank Conflict

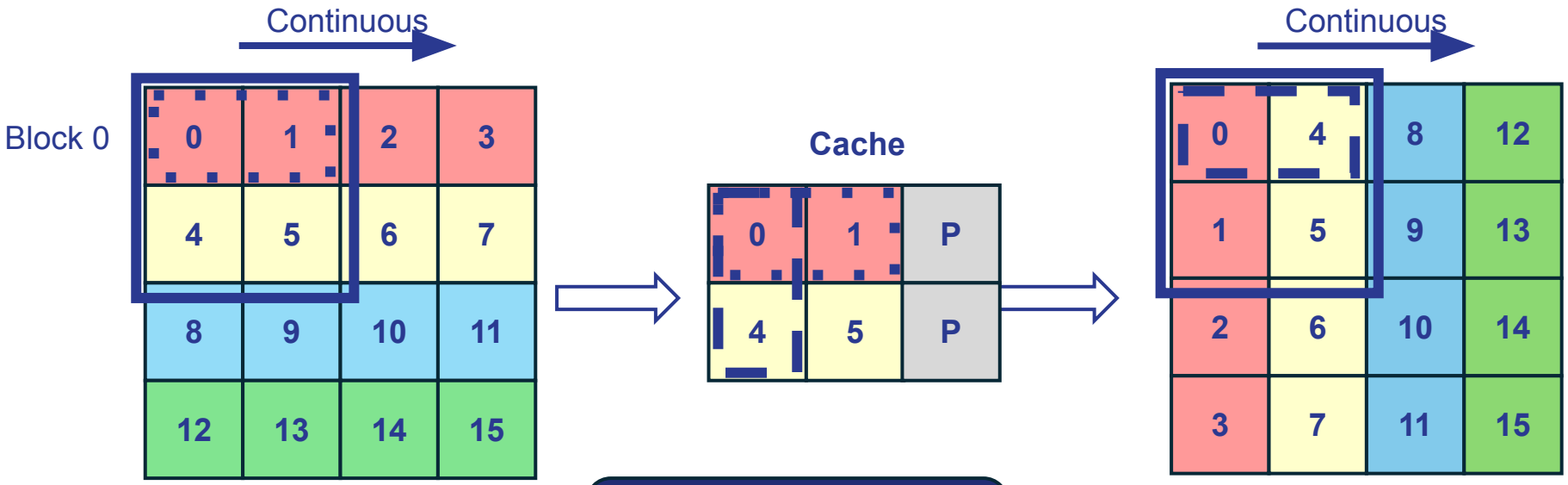


Bank Conflict

0	1	2	3	P
4	5	6	7	P
8	9	10	11	P
12	13	14	15	P

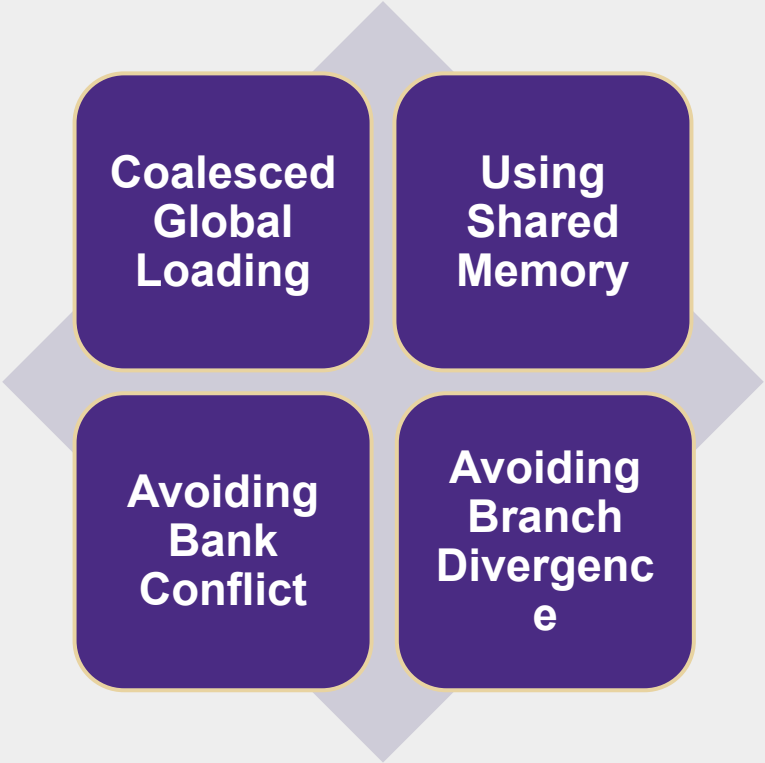


Transpose V4: Padding



280 us
>1.9TB/s

More complex problem: Reduction



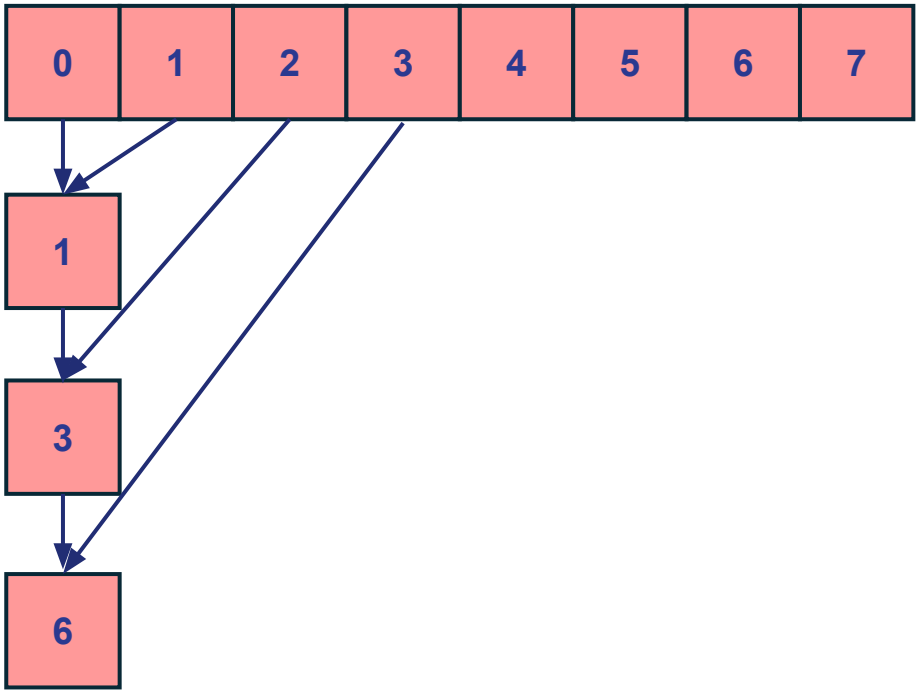
Reduction Problem

- Reduction example:

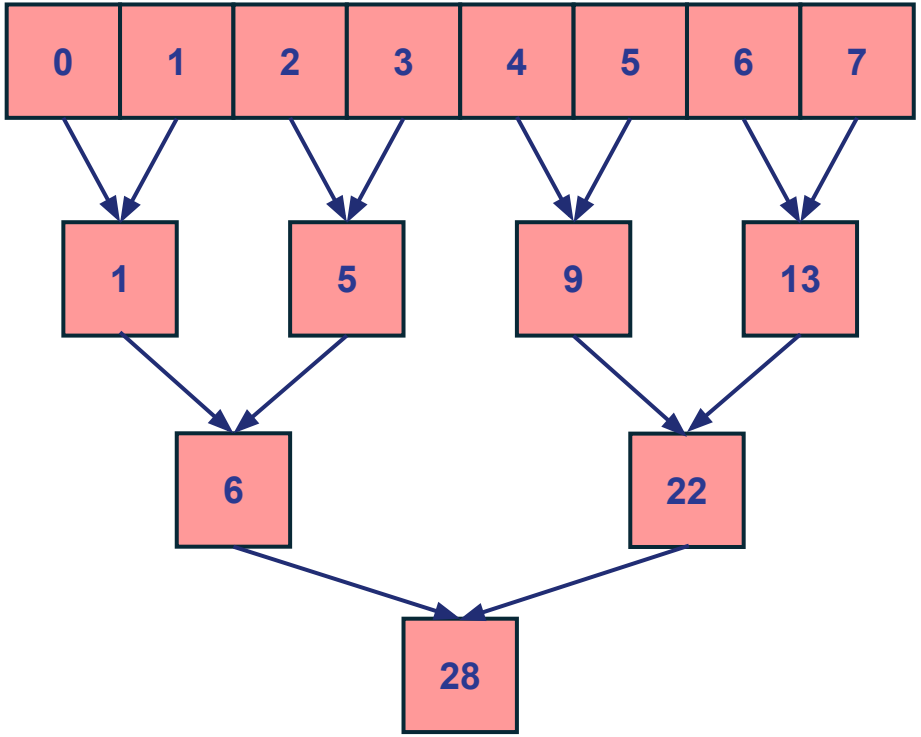
  for elements in array:

  Temp = op (temp, element)

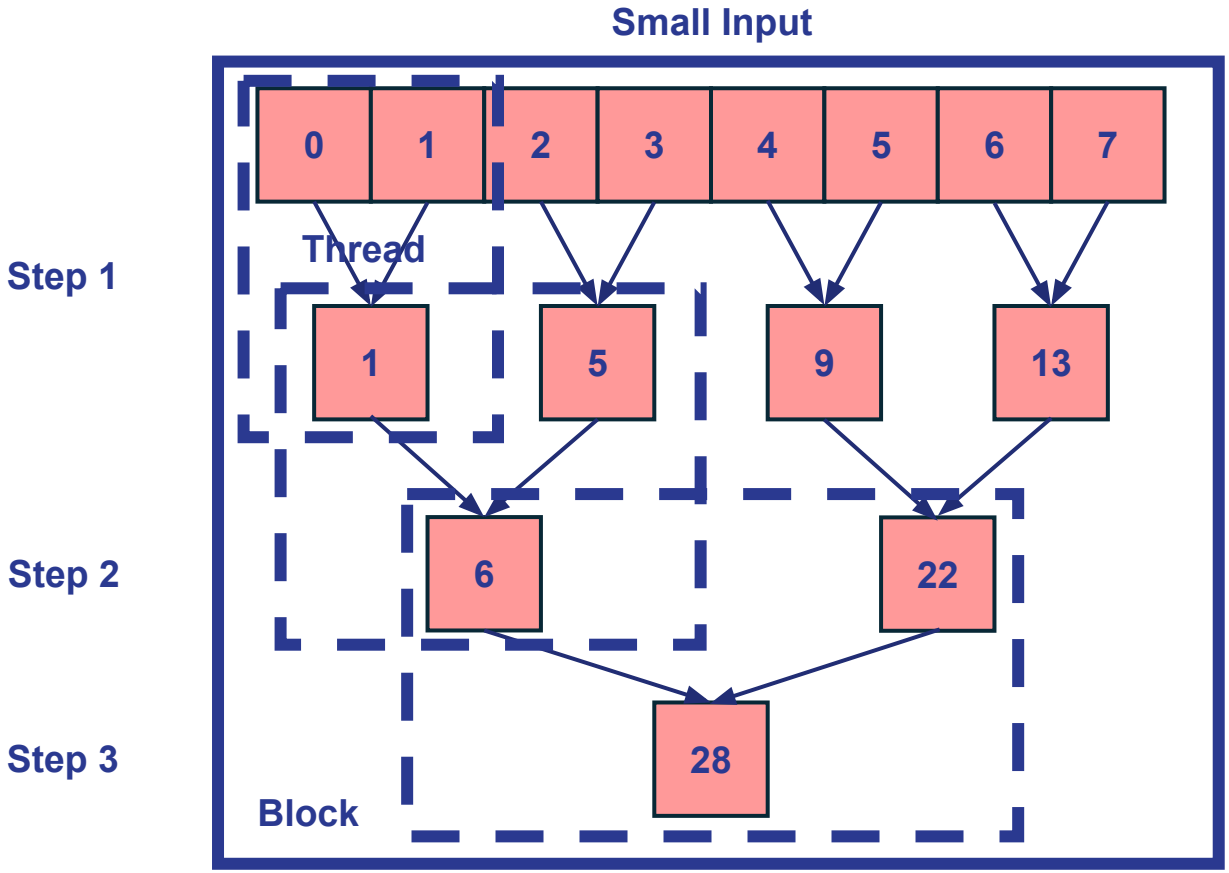
- Examples of reduction
  - Sum
  - Max / Min
  - …



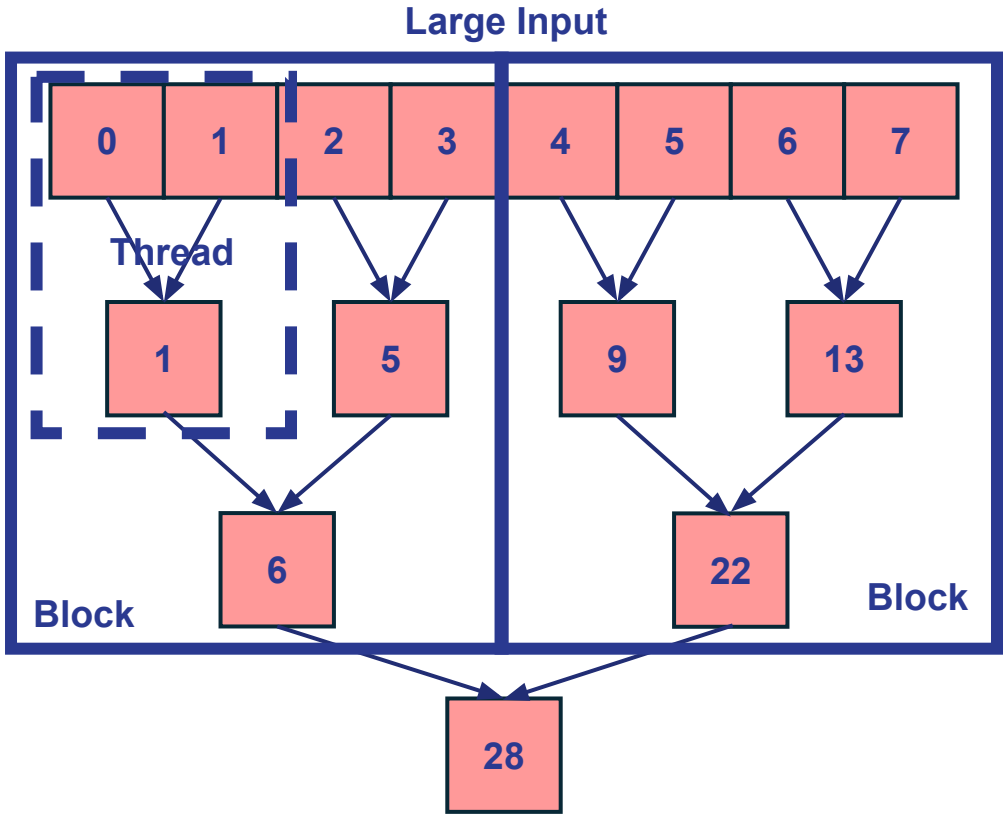
Parallel Reduction



Global View of the Reduction Kernel



Global View of the Reduction Kernel



Reduction #1

- Step i
  - threadID % 2^N == 0 does the work
  - stride = 2 ^ (N-1)

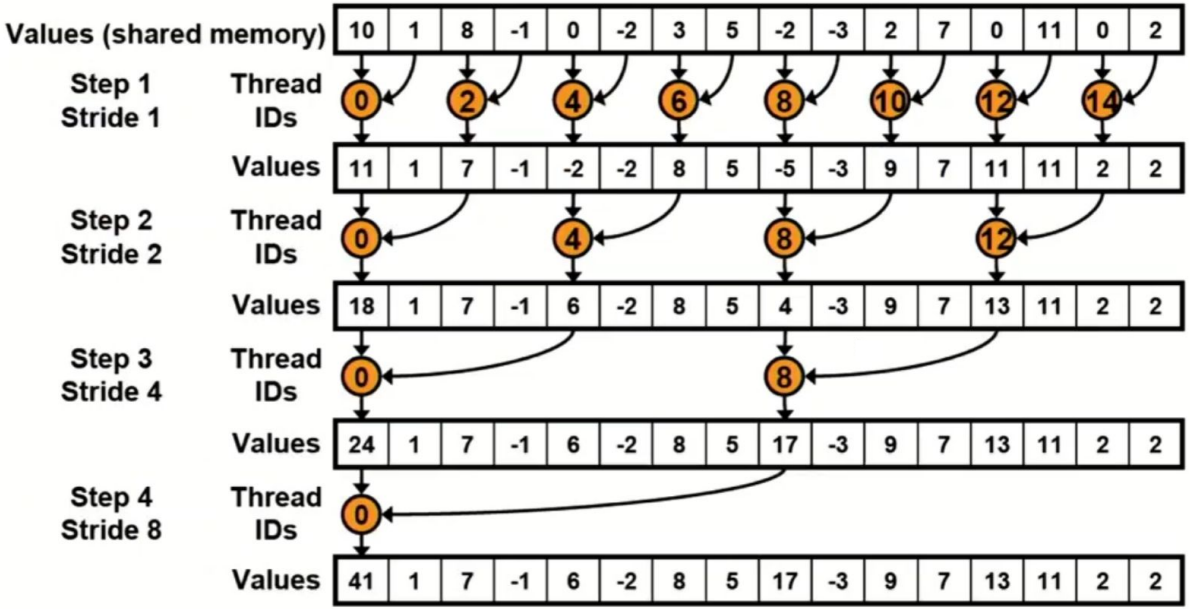
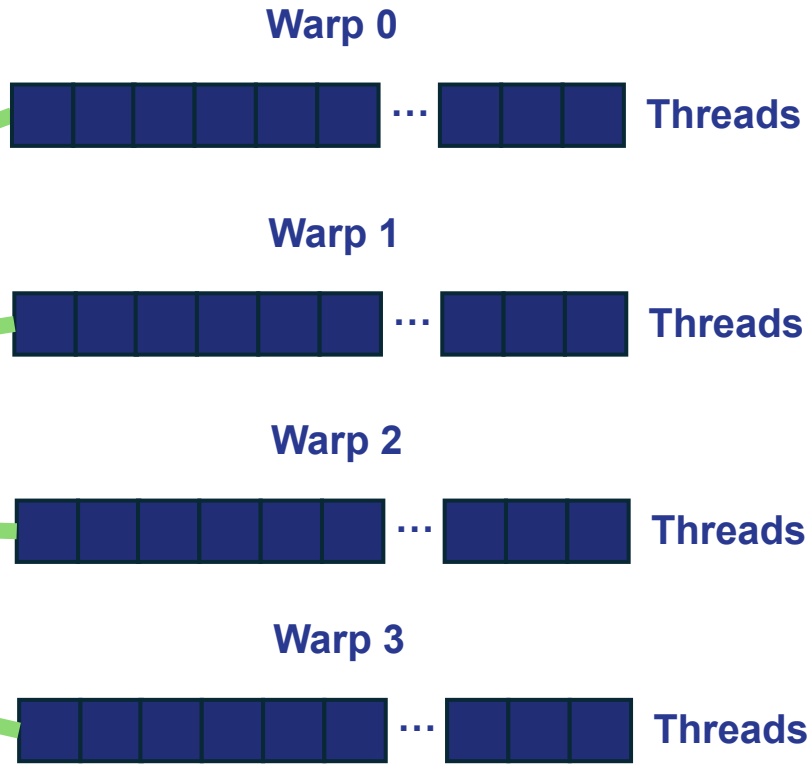


Figure taken from Caltech CS 179: GPU Programming Lecture 7

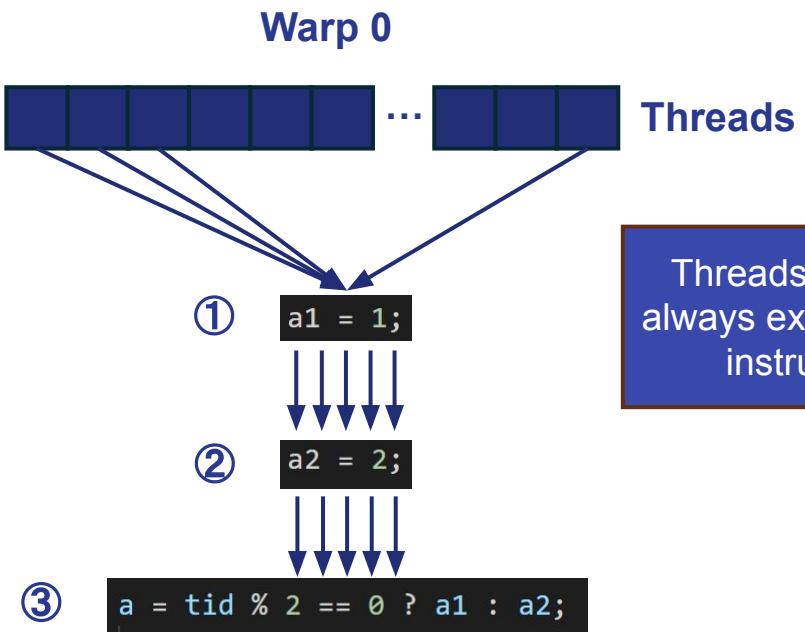
Handling Branches in CUDA

```cuda
__global__ void kernel() {
    int tid = threadIdx.x;
    int a;
    if (tid < 32) {
        a = 1;
    } else if (tid < 64) {
        a = 2;
    } else if (tid < 96) {
        a = 3;
    } else {
        a = 4;
    }
}
```



Handling Branches in CUDA

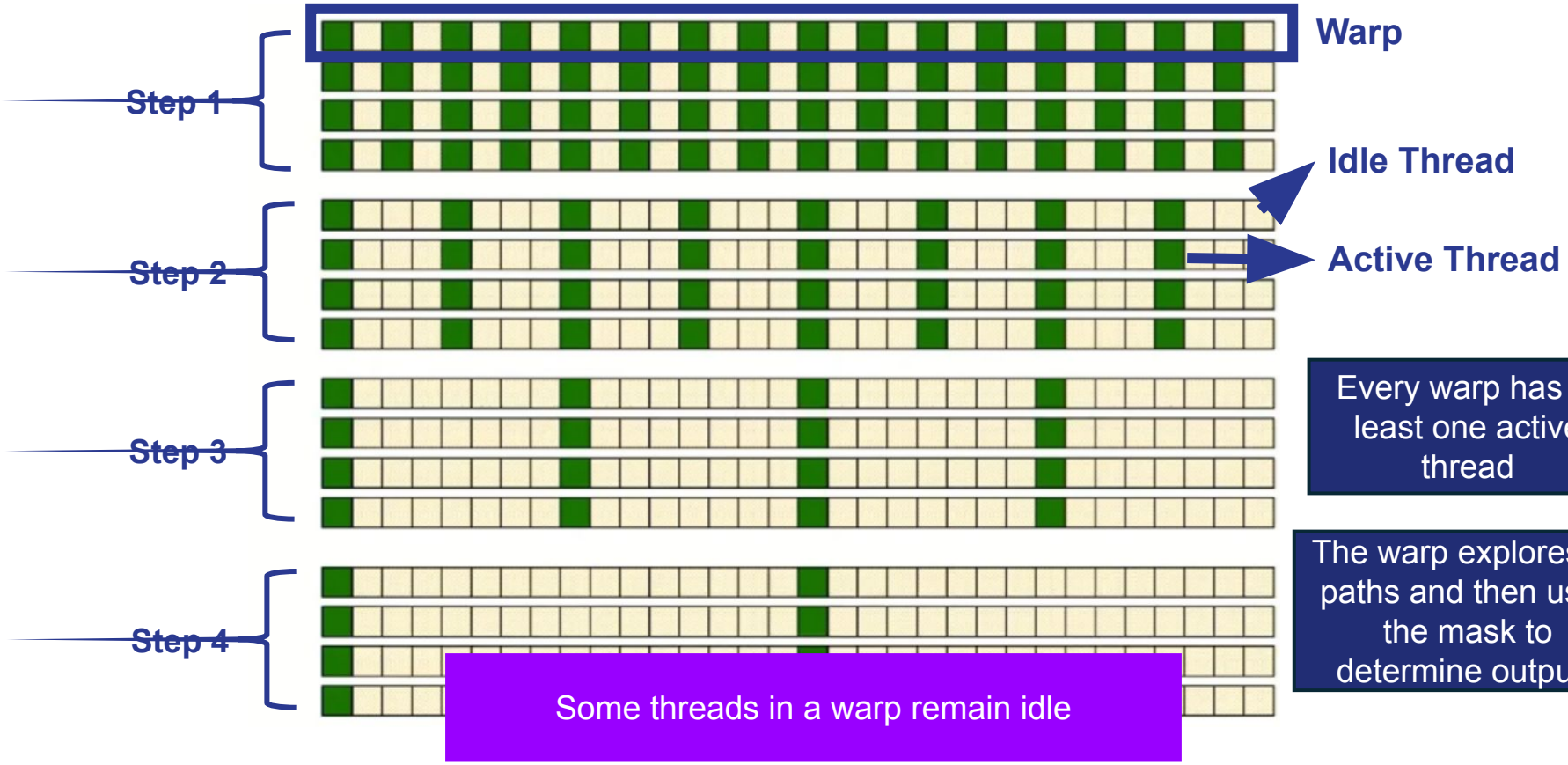
```cpp
__global__ void kernel2() {
    int tid = threadIdx.x;
    int a;
    if (tid % 2 == 0) {
        a = 1;
    } else {
        a = 2;
    }
}
```



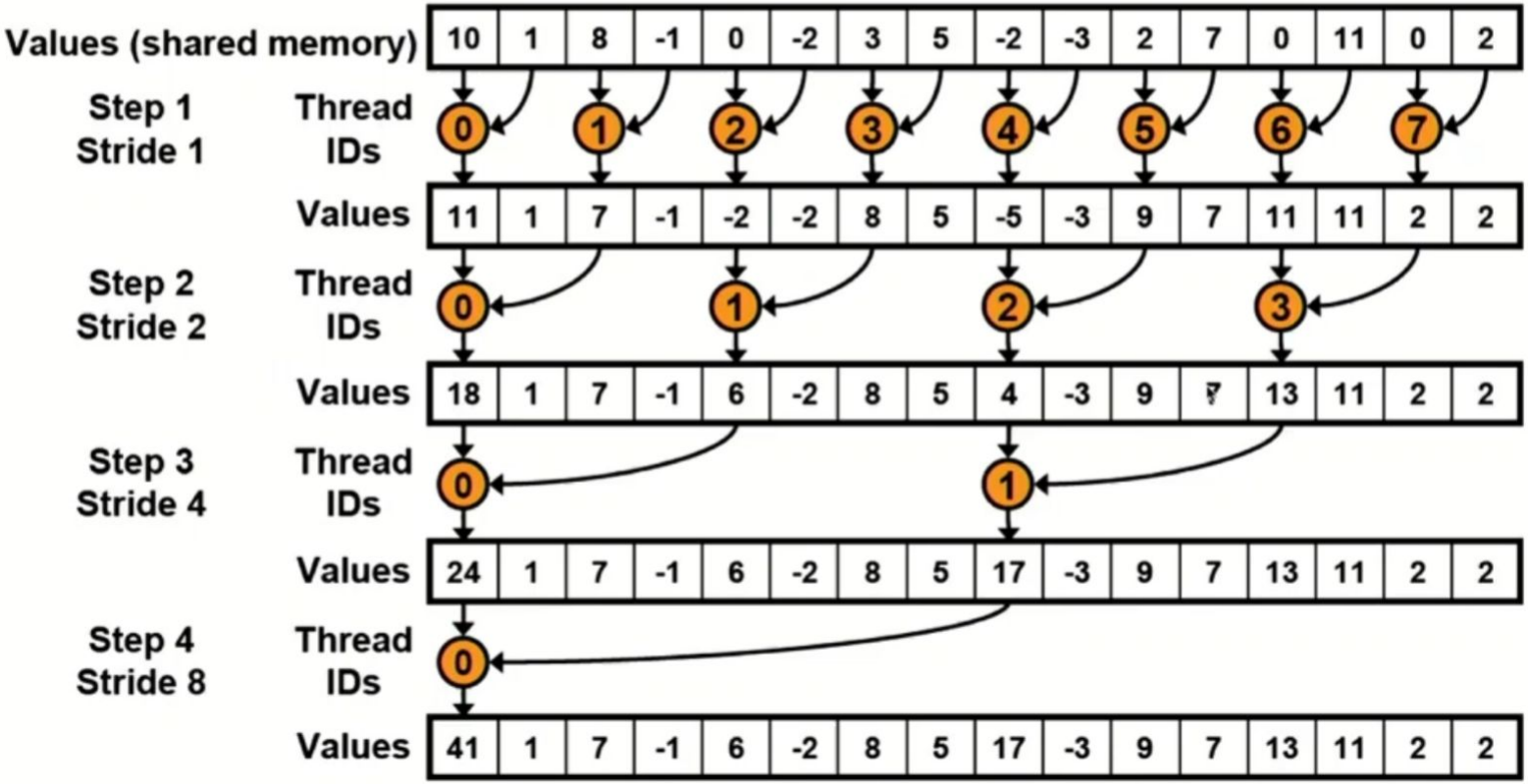
Threads in a warp always execute same instructions

Explore all paths and then use mask to determine outputs

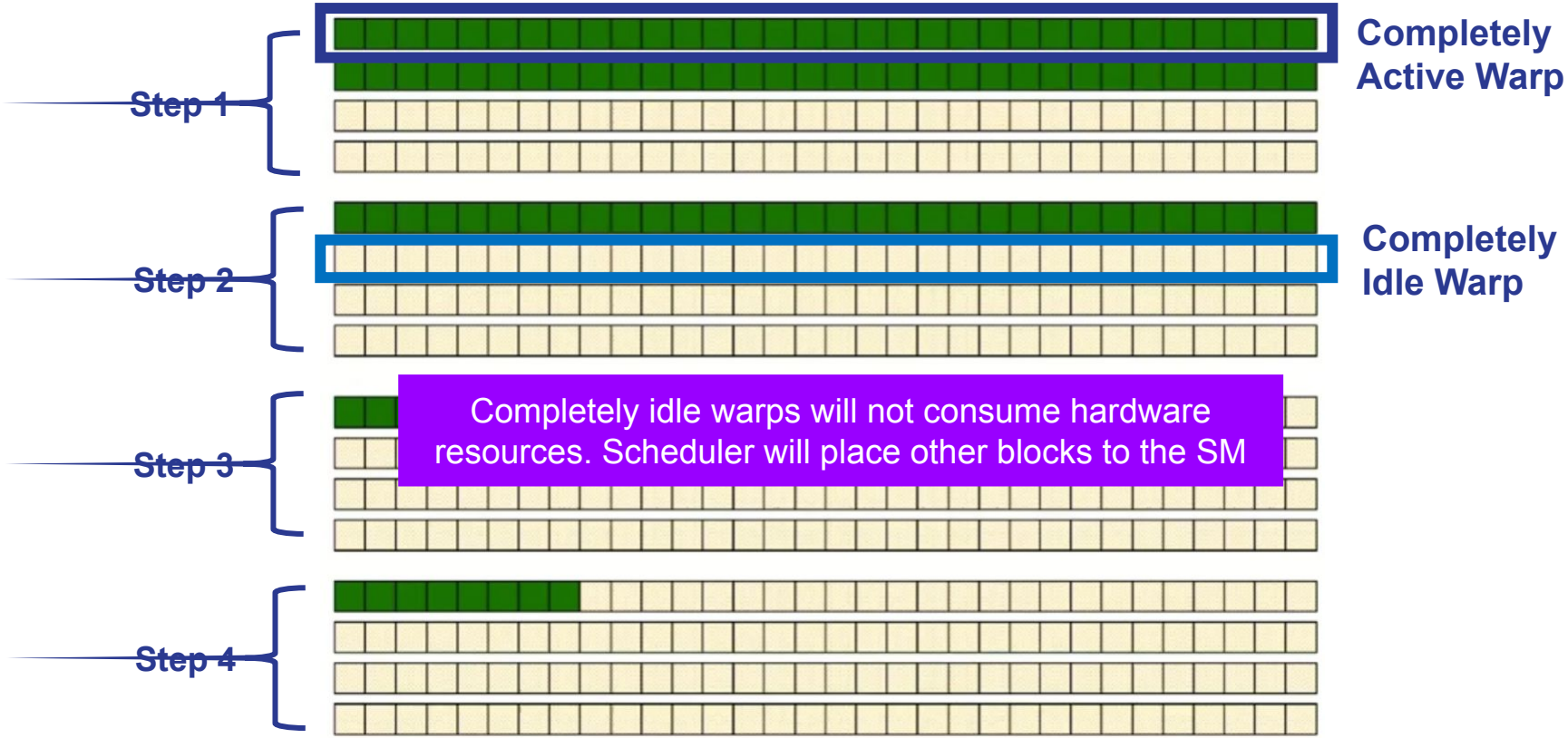
Branch Divergence



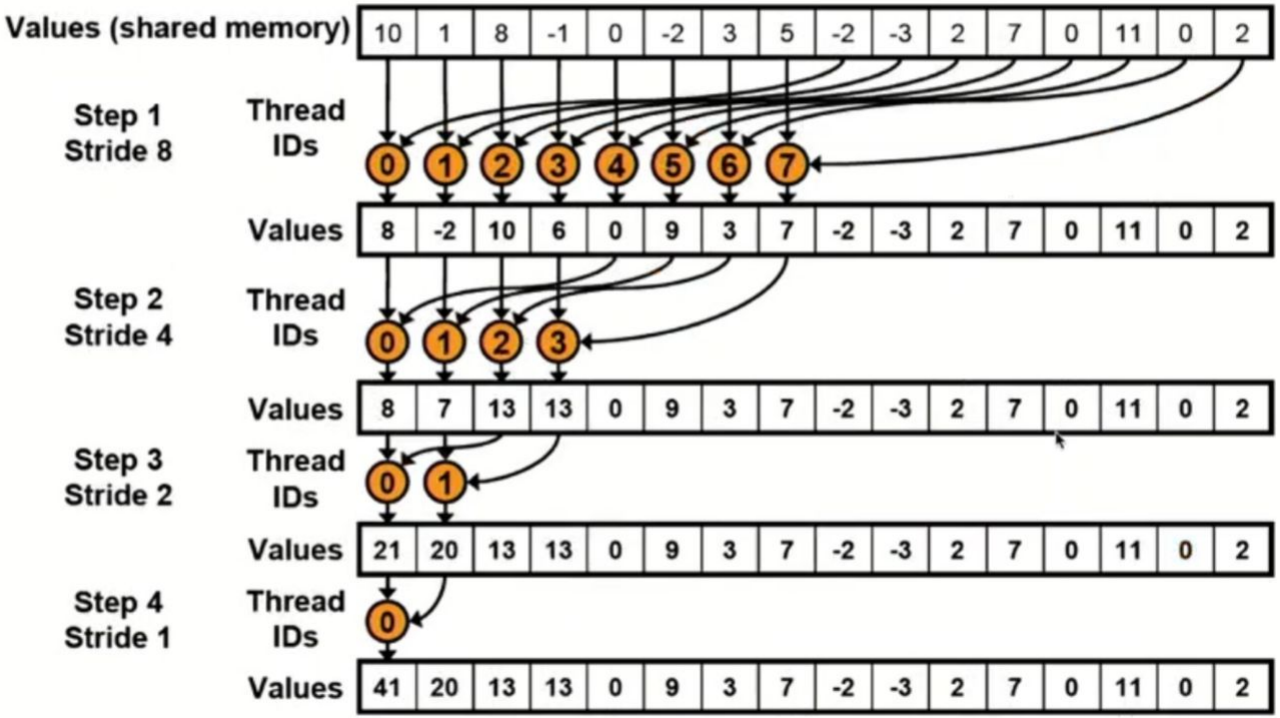
Reduction #2: Access Pattern



Resolved Divergence

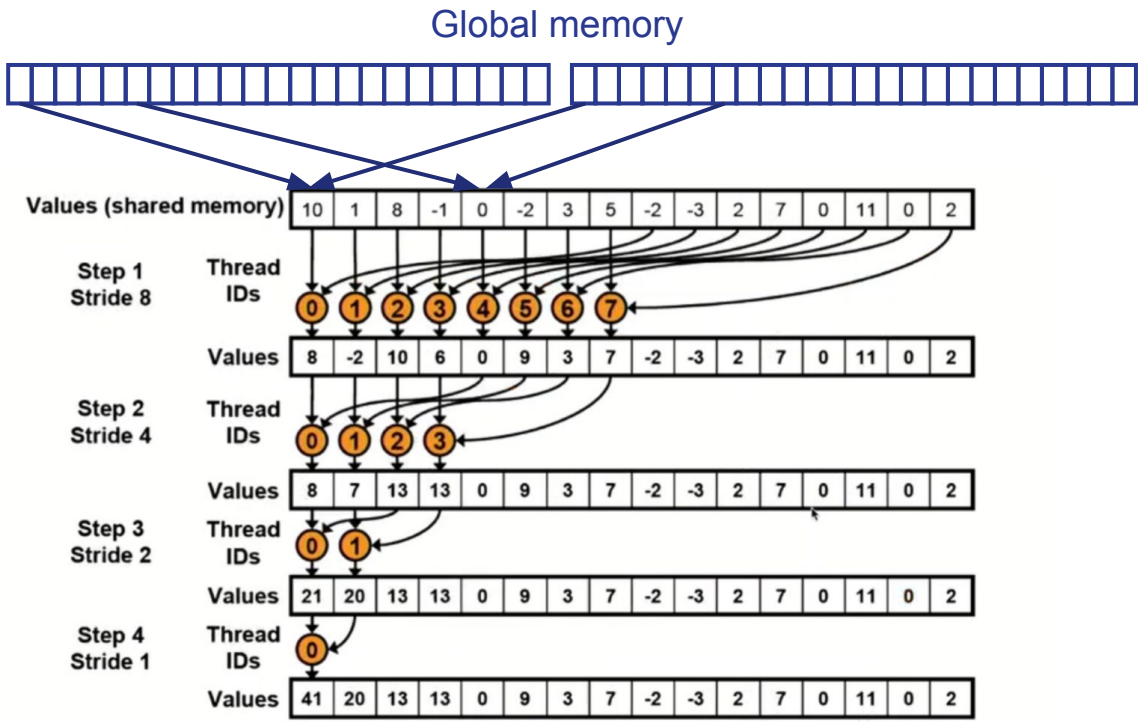


Reduction #3: Sequential Accesses

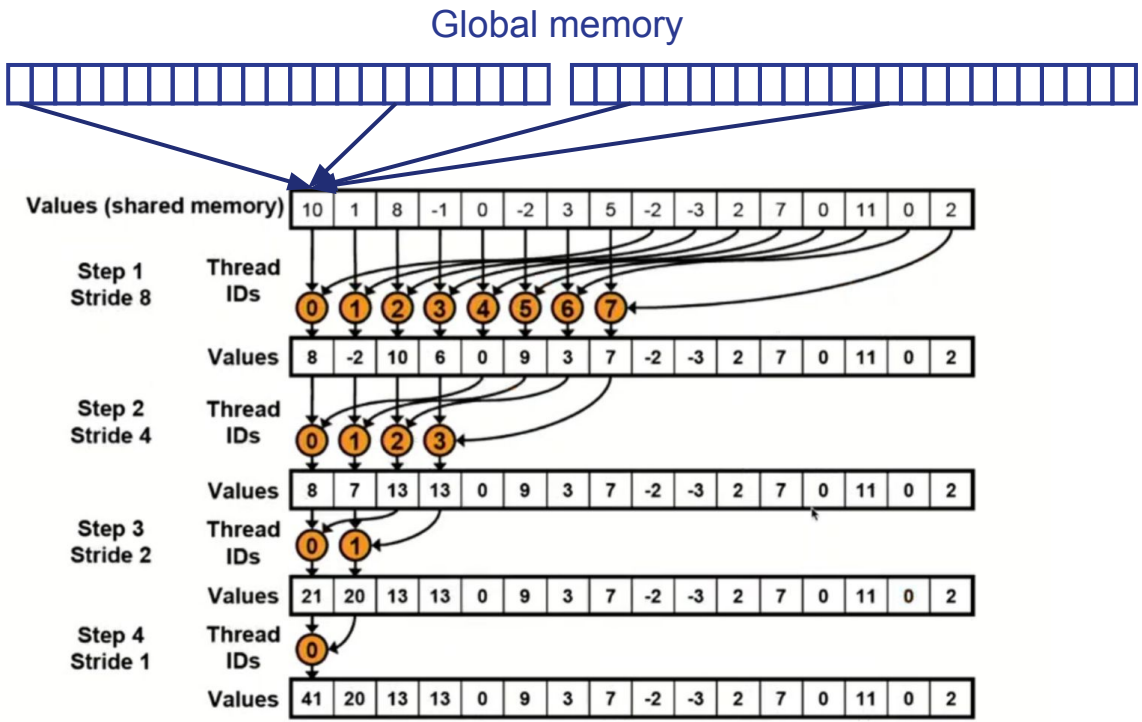


Eliminates bank conflicts

Reduction #4: Load two elements



Reduction #5: Load more elements



Trade-off: Efficiency and SM occupancy

- Loading more elements means higher memory utilization
- However, the number of blocks reduces, and GPU utilization goes down
  - If your number of blocks are lower than the number of SMs, that’s another form of waste
  - You need to profile and find the sweet spot

Takeaways

- GPU code is hard to write
- The code we write is extremely tailored to our architecture
- CPUs are lucky to have nice abstractions
  - Read: go to considered harmful

Additional CUDA features in modern GPUs

- Unified memory address (P100+)
- NvLink (P100+)
- Clusters (H100+)
- TMA (H100+)
- NVSHARP (H100+)
- FP4 and FP6 (B100+)

Optional Reading

https://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/pages/lecture-slides/

peterson's algorithm
http://ocw.mit.edu/courses/6-172-performance-engineering-of-software-systems-fall-2018/resources/mit6_172f18_lec17/